This method uses two very useful string methods: downcase returns a lowercase version of a string, and scan returns an array of substrings that match a given pattern. In this case, the pattern is [\w']+, which matches sequences containing "word characters" and single quotes.

We can play with this method. Notice how the result is an array, and notice that the words are in lowercase and the punctuation is gone:

```
p words_from_string("I like Ruby, it is (usually) optimized for programmer happiness")
```

produces:

```
["i", "like", "ruby", "it", "is", "usually", "optimized", "for", "programmer",
"happiness"]
```

Our next task is to calculate word frequencies. To do this, we'll create a hash object indexed by the words in our list. Each entry in this hash stores the number of times that word occurred. Let's say we already have read part of the list, and we have seen the word *the* already. Then we'd have a hash that contained this:

```
{...,  "the" => 1, ...}
```

If the variable next_word contained the word *the*, then incrementing the count is as simple as this:

```
counts[next_word] += 1
```

We'd then end up with a hash containing the following:

```
{...,  "the" => 2, ...}
```

Our only problem is what to do when we encounter a word for the first time. We'll try to increment the entry for that word, but there won't be one, so our program will fail. There are a number of solutions to this. One is to check to see whether the entry exists before doing the increment:

```
if counts.has_key?(next_word)
  counts[next_word] += 1
else
  counts[next_word] = 1
end
```

However, there's a tidier way. If we create a hash object using Hash.new(0), the parameter, 0 in this case, will be used as the hash's default value—it will be the value returned if you look up a key that isn't yet in the hash. Using that, we can write our count_frequency method:

tut_containers/word_freq/count_frequency.rb

```
def count_frequency(word_list)
  counts = Hash.new(0)
  word_list.each do |word|
    counts[word] += 1
  end
  counts
end

p count_frequency(["sparky", "the", "cat", "sat", "on", "the", "mat"])
```

produces:

```
{"sparky"=>1, "the"=>2, "cat"=>1, "sat"=>1, "on"=>1, "mat"=>1}
```